

Day 58 -- Line-by-Line Syntax Explanation

day58_rnn_bppt.py, segment by segment | MD Mutasim Billah

1. rnn_backward() -- the Whole of BPTT

accumulators that start at zero

```
dwxh = np.zeros((1, Whh.shape[0])) # receives one contribution PER timestep
dWhh = np.zeros_like(Whh)
dbh = np.zeros(Whh.shape[0])
dh = dh_last # gradient arriving at the LAST hidden state
```

The three shared-parameter gradients start at zero because they will be built up piece by piece, one timestep at a time. dh starts as the gradient arriving from the output layer -- in a many-to-one network, that's the only place gradient enters the recurrence.

one backward step through time

```
for t in reversed(range(T)): # T-1, T-2, ..., 0
    h_prev = H[:, t - 1, :] if t > 0 else h0 # the state that FED this timestep
    da_t = dh * (1.0 - h_t ** 2) # undo tanh, from the cached output
    dwxh += x_t.T @ da_t / m # ACCUMULATE
    dWhh += h_prev.T @ da_t / m # ACCUMULATE -- h_prev, not h_t
    dbh += da_t.sum(axis=0) / m # ACCUMULATE
    dh = da_t @ Whh.T # hand gradient back to h_{t-1}
```

reversed(range(T)) walks time backward. The h_prev line handles the first timestep, whose 'previous state' is h0 rather than a stored H entry. The /m divides by batch size, matching every weight gradient in this series. The final line OVERWRITES dh (not +=) because in a many-to-one network no other gradient enters h_{t-1} -- a many-to-many network would add a per-timestep output gradient here.

2. The Gradient Check's Restore Step

undoing backward()'s built-in SGD update

```
net.backward(y_oh) # computes grads AND applies an SGD step
for name in ["Wxh", "Whh", "bh", "why", "by"]:
    getattr(net, name)[...] += net.lr * net.grads[name] # p -= lr*g, so p += lr*g restores p
```

backward() both computes gradients and updates the weights. The check must perturb the SAME weights the gradients were computed at, so the update is reversed first. The [...] makes the += modify the existing array in place rather than rebinding a new one -- the lesson Day 53 learned the hard way.

3. Two Kinds of Sensitivity

small-signal vs. large-signal

```
# small-signal: a local derivative, eps = 1e-3
Xp = X.copy(); Xp[0, 0] += eps
return np.linalg.norm(hp - hm) / (2 * eps)

# large-signal: a full task-sized spike, no division by eps
Xs = X.copy(); Xs[:, 0] += 3.0
return np.linalg.norm(h_spike - h_plain, axis=1).mean()
```

The first divides by $2 \cdot \text{eps}$, estimating a derivative -- meaningful only for infinitesimal changes. The second applies a change the size of the task's actual signal and measures the raw difference. For a linear system they would agree up to scale. They disagree here by orders of magnitude, and in opposite directions after training -- the fingerprint of a strongly nonlinear, saturating mechanism.